```
void undefined_behaviour_with_double_checked_locking()
{
    if(!resource_ptr)                                    ❶
    {
        std::lock_guard<std::mutex> lk(resource_mutex);
        if(!resource_ptr)                                ❷
        {
            resource_ptr.reset(new some_resource);       ❸
        }
    }
    resource_ptr->do_something();                        ❹
}
```

Unfortunately, this pattern is infamous for a reason: it has the potential for nasty race conditions, because the read outside the lock ❶ isn't synchronized with the write done by another thread inside the lock ❸. This therefore creates a race condition that covers not just the pointer itself but also the object pointed to; even if a thread sees the pointer written by another thread, it might not see the newly created instance of `some_resource`, resulting in the call to `do_something()` ❹ operating on incorrect values. This is an example of the type of race condition defined as a *data race* by the C++ Standard and thus specified as *undefined behavior*. It's is therefore quite definitely something to avoid. See chapter 5 for a detailed discussion of the memory model, including what constitutes a *data race*.

The C++ Standards Committee also saw that this was an important scenario, and so the C++ Standard Library provides `std::once_flag` and `std::call_once` to handle this situation. Rather than locking a mutex and explicitly checking the pointer, every thread can just use `std::call_once`, safe in the knowledge that the pointer will have been initialized by some thread (in a properly synchronized fashion) by the time `std::call_once` returns. Use of `std::call_once` will typically have a lower overhead than using a mutex explicitly, especially when the initialization has already been done, so should be used in preference where it matches the required functionality. The following example shows the same operation as listing 3.11, rewritten to use `std::call_once`. In this case, the initialization is done by calling a function, but it could just as easily have been done with an instance of a class with a function call